

Viterbi algorithm

The **Viterbi algorithm** is a dynamic programming algorithm for finding the most likely sequence of hidden states—called the **Viterbi path**—that results in a sequence of observed events, especially in the context of Markov information sources and hidden Markov models.

The algorithm has found universal application in decoding the convolutional codes used in both CDMA and GSM digital cellular, dial-up modems, satellite, deep-space communications, and 802.11 wireless LANs. It is now also commonly used in speech recognition, speech synthesis, diarization,^[1] keyword spotting, computational linguistics, and bioinformatics. For example, in speech-to-text (speech recognition), the acoustic signal is treated as the observed sequence of events, and a string of text is considered to be the "hidden cause" of the acoustic signal. The Viterbi algorithm finds the most likely string of text given the acoustic signal.

Contents

History

Extensions

Pseudocode

Example

See also

References

General references

Implementations

External links

History

The Viterbi algorithm is named after Andrew Viterbi, who proposed it in 1967 as a decoding algorithm for convolutional codes over noisy digital communication links.^[2] It has, however, a history of multiple invention, with at least seven independent discoveries, including those by Viterbi, Needleman and Wunsch, and Wagner and Fischer.^[3]

"Viterbi path" and "Viterbi algorithm" have become standard terms for the application of dynamic programming algorithms to maximization problems involving probabilities.^[3] For example, in statistical parsing a dynamic programming algorithm can be used to discover the single most likely context-free derivation (parse) of a string, which is commonly called the "Viterbi parse".^{[4][5][6]} Another application is in target tracking, where the track is computed that assigns a maximum likelihood to a sequence of observations.^[7]

Extensions

A generalization of the Viterbi algorithm, termed the *max-sum algorithm* (or *max-product algorithm*) can be used to find the most likely assignment of all or some subset of latent variables in a large number of graphical models, e.g. Bayesian networks, Markov random fields and conditional random fields. The latent variables need in general to be connected in a

way somewhat similar to an HMM, with a limited number of connections between variables and some type of linear structure among the variables. The general algorithm involves *message passing* and is substantially similar to the belief propagation algorithm (which is the generalization of the forward-backward algorithm).

With the algorithm called iterative Viterbi decoding one can find the subsequence of an observation that matches best (on average) to a given hidden Markov model. This algorithm is proposed by Qi Wang et al. to deal with turbo code.^[8] Iterative Viterbi decoding works by iteratively invoking a modified Viterbi algorithm, reestimating the score for a filler until convergence.

An alternative algorithm, the Lazy Viterbi algorithm, has been proposed.^[9] For many applications of practical interest, under reasonable noise conditions, the lazy decoder (using Lazy Viterbi algorithm) is much faster than the original Viterbi decoder (using Viterbi algorithm). While the original Viterbi algorithm calculates every node in the trellis of possible outcomes, the Lazy Viterbi algorithm maintains a prioritized list of nodes to evaluate in order, and the number of calculations required is typically fewer (and never more) than the ordinary Viterbi algorithm for the same result. However, it is not so easy to parallelize in hardware.

Pseudocode

This algorithm generates a path $\mathbf{X} = (x_1, x_2, \dots, x_T)$, which is a sequence of states $x_n \in \mathcal{S} = \{s_1, s_2, \dots, s_K\}$ that generate the observations $\mathbf{Y} = (y_1, y_2, \dots, y_T)$ with $y_n \in \mathcal{O} = \{o_1, o_2, \dots, o_N\}$ (N being the count of observations (observation space, see below)).

Two 2-dimensional tables of size $K \times T$ are constructed:

- Each element $T_1[i, j]$ of T_1 stores the probability of the most likely path so far $\hat{\mathbf{X}} = (\hat{x}_1, \hat{x}_2, \dots, \hat{x}_j)$ with $\hat{x}_j = s_i$ that generates $\mathbf{Y} = (y_1, y_2, \dots, y_j)$.
- Each element $T_2[i, j]$ of T_2 stores $\hat{x}_{j-1}$ of the most likely path so far $\hat{\mathbf{X}} = (\hat{x}_1, \hat{x}_2, \dots, \hat{x}_{j-1}, \hat{x}_j = s_i)$ for $\forall j, 2 \leq j \leq T$

The table entries $T_1[i, j], T_2[i, j]$ are filled by increasing order of $K \cdot j + i$.

$$T_1[i, j] = \max_k (T_1[k, j-1] \cdot A_{ki} \cdot B_{iy_j}), \text{ and}$$

$$T_2[i, j] = \operatorname{argmax}_k (T_1[k, j-1] \cdot A_{ki} \cdot B_{iy_j}),$$

with A_{ki} and B_{iy_j} as defined below. Note that B_{iy_j} does not need to appear in the latter expression, as it's non-negative and independent of k and thus does not affect the argmax .

INPUT

- The observation space $\mathcal{O} = \{o_1, o_2, \dots, o_N\}$
- the state space $\mathcal{S} = \{s_1, s_2, \dots, s_K\}$
- an array of initial probabilities $\Pi = (\pi_1, \pi_2, \dots, \pi_K)$ such that π_i stores the probability that $x_1 == s_i$
- a sequence of observations $\mathbf{Y} = (y_1, y_2, \dots, y_T)$ such that $y_t == i$ if the observation at time t is o_i
- transition matrix $\mathbf{A}$ of size $K \times K$ such that A_{ij} stores the transition probability of transiting from state s_i to state s_j
- emission matrix $\mathbf{B}$ of size $K \times N$ such that B_{ij} stores the probability of observing o_j from state s_i

OUTPUT

- The most likely hidden state sequence $\mathbf{X} = (x_1, x_2, \dots, x_N)$

EXPLANATION

Suppose we are given a hidden Markov model (HMM) with state space $\mathcal{S}$, initial probabilities π_i of being in state i and transition probabilities $a_{i,j}$ of transitioning from state i to state j . Say we observe outputs $y_1, \dots, y_T$. The most likely state sequence $x_1, \dots, x_T$ that produces the observations is given by the recurrence relations:^[10]

$$\begin{aligned} V_{1,k} &= P(y_1 \mid k) \cdot \pi_k \\ V_{t,k} &= \max_{x \in \mathcal{S}} (P(y_t \mid k) \cdot a_{x,k} \cdot V_{t-1,x}) \end{aligned}$$

Here $V_{t,k}$ is the probability of the most probable state sequence $P(x_1, \dots, x_t, y_1, \dots, y_t)$ responsible for the first t observations that have k as its final state. The Viterbi path can be retrieved by saving back pointers that remember which state x was used in the second equation. Let $\text{Ptr}(k, t)$ be the function that returns the value of x used to compute $V_{t,k}$ if $t > 1$, or k if $t = 1$. Then:

$$\begin{aligned} x_T &= \arg \max_{x \in \mathcal{S}} (V_{T,x}) \\ x_{t-1} &= \text{Ptr}(x_t, t) \end{aligned}$$

Here we're using the standard definition of arg max.

The complexity of this implementation is $O(T \times |\mathcal{S}|^2)$. A better estimation exists if the maximum in the internal loop is instead found by iterating only over states that directly link to the current state (i.e. there is an edge from k to j). Then using amortized analysis one can show that the complexity is $O(T \times (|\mathcal{S}| + |\mathcal{E}|))$, where $\mathcal{E}$ is the number of edges in the graph.

Example

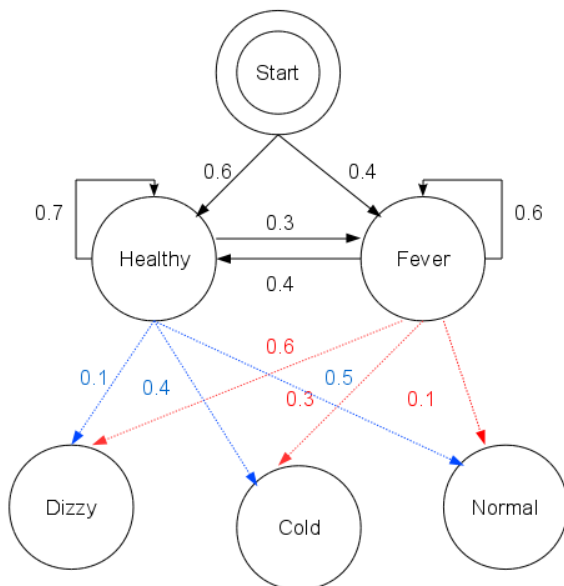
Consider a village where all villagers are either healthy or have a fever and only the village doctor can determine whether each has a fever. The doctor diagnoses fever by asking patients how they feel. The villagers may only answer that they feel normal, dizzy, or cold.

The doctor believes that the health condition of his patients operate as a discrete Markov chain. There are two states, "Healthy" and "Fever", but the doctor cannot observe them directly; they are *hidden* from him. On each day, there is a certain chance that the patient will tell the doctor he/she is "normal", "cold", or "dizzy", depending on their health condition.

The *observations* (normal, cold, dizzy) along with a *hidden* state (healthy, fever) form a hidden Markov model (HMM), and can be represented as follows in the Python programming language:

```
obs = ('normal', 'cold', 'dizzy')
states = ('Healthy', 'Fever')
start_p = {'Healthy': 0.6, 'Fever': 0.4}
trans_p = {
    'Healthy' : {'Healthy': 0.7, 'Fever': 0.3},
    'Fever' : {'Healthy': 0.4, 'Fever': 0.6}
}
emit_p = {
    'Healthy' : {'normal': 0.5, 'cold': 0.4, 'dizzy': 0.1},
    'Fever' : {'normal': 0.1, 'cold': 0.3, 'dizzy': 0.6}
}
```

In this piece of code, `start_probability` represents the doctor's belief about which state the HMM is in when the patient first visits (all he knows is that the patient tends to be healthy). The particular probability distribution used here is not the equilibrium one, which is (given the transition probabilities) approximately `{'Healthy': 0.58, 'Fever': 0.42}`. The `transition_probability` represents the change of the health condition in the underlying Markov chain. In this example, there is only a 30% chance that tomorrow the patient will have a fever if he is healthy today. The `emission_probability` represents how likely each possible observation, normal, cold, or dizzy is given their underlying condition, healthy or fever. If the patient is healthy, there is a 50% chance that he feels normal; if he has a fever, there is a 60% chance that he feels dizzy.

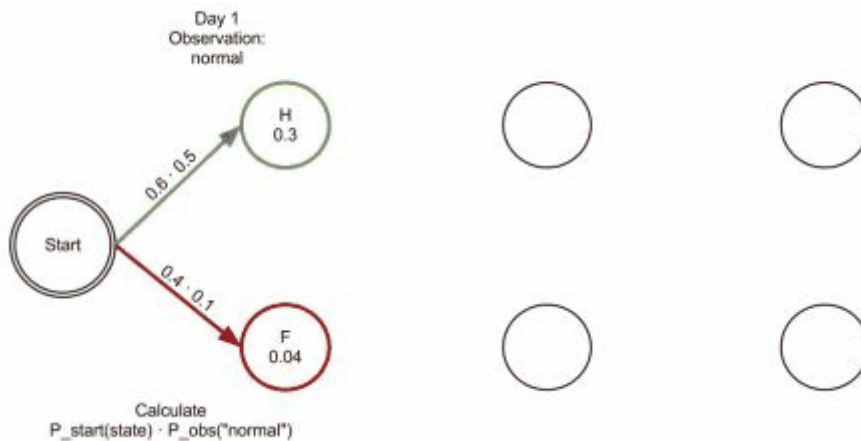


Graphical representation of the given HMM

The patient visits three days in a row and the doctor discovers that on the first day he feels normal, on the second day he feels cold, on the third day he feels dizzy. The doctor has a question: what is the most likely sequence of health conditions of the patient that would explain these observations? This is answered by the Viterbi algorithm.

This reveals that the observations ['normal', 'cold', 'dizzy'] were most likely generated by states ['Healthy', 'Healthy', 'Fever']. In other words, given the observed activities, the patient was most likely to have been healthy both on the first day when he felt normal as well as on the second day when he felt cold, and then he contracted a fever the third day.

The operation of Viterbi's algorithm can be visualized by means of a trellis diagram. The Viterbi path is essentially the shortest path through this trellis. The trellis for the clinic example is shown below; the corresponding Viterbi path is in bold:



Animation of the trellis diagram for the Viterbi algorithm. After Day 3, the most likely path is ['Healthy', 'Healthy', 'Fever']

Retrieved from "https://en.wikipedia.org/w/index.php?title=Viterbi_algorithm&oldid=867573879"

This page was last edited on 6 November 2018, at 16:14 (UTC).

Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.